

Parallel Course

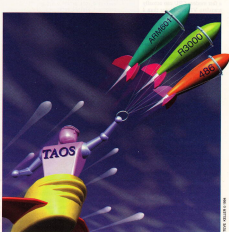
DICK POUNTAIN

Nowadays several computers can generate realistic ray-traced images, and some of them can even do it fast. A rendering demonstration I recently witnessed was impressive for one special reason: The PC doing the rendering was executing the same ray-tracing program in parallel on an Intel 486, four James T860 transputers, and four Mips R3000s. What made this feat possible was Taos (pronounced "dawse," as in the Chinese philosophy Taoism), a radically different, object-oriented parallel operating system.

Roots of Taos

Most operating systems are created either by large computer companies or by university researchers, but Taos is different—it's the product of a devoted group of enthusiasts with an idea that was well ahead of its time. The principal architect of Taos is Chris Hinsley, formerly a professional games programmer with his titles for the Atari ST and the Commodore Amiga to his credit. Although writing solely in assembly language, Hinsley devised his own object-oriented development style based on macros, which sparked the original idea for Taos. Fired by the launch of Iroquois's transputer architecture, Hinsley wanted to create a real-time operating system that could harness the parallel-processing power he believed would be needed for future multimedia systems.

When I first wrote about Taos in the International edition of the March 1991 BYTE, it seemed outrageously removed from the mainstream. Today, many of the concepts it is built on are showing up in the commercial operating-system world. Everyone wants a microkernel now, but Taos is already a nanokernel system, with its tiny 12-KB kernel running on each processor in a parallel network. Taligent promises objects from



IBM: S. KELLER © IBM

The Taos operating system uses objects from the ground up to enable processors based on different architectures to work together on the same problem

the ground up with dynamic binding; Taos has had them from the start.

One difference is Taos doesn't really aspire to mainstream desktop status; instead, it is a fast and skinny system for embedded applications. Tao Systems is promoting it into the multimedia and games console markets.

The Virtual Processor

By far, the most radical aspect of Taos is its hardware independence. Taos programs are all written in the machine code of a VP (virtual processor), which is called VPcode. The Taos kernel translates VPcode into the native machine code of each

real processor immediately before running it—there is little or no run-time penalty, unlike earlier interpreted systems (e.g., the UCSD p-system), which were slow. Taos's fine-grained object orientation and dynamic binding makes this translation strategy feasible: VPcode modules are always small (typically a few

hundred bytes), so they can be translated on the fly as they load from disk into memory. Huge monolithic applications like Excel or WordPerfect wouldn't lend themselves to this approach. But Tao Systems' translator supremo Andy Hanson stressed to me that a fast modern CPU can actually translate VPcode faster than a hard drive can transfer data.

The imaginary VP is a 32-bit little-endian RISC machine with 16 registers. It supports data types from 8-bit bytes up to 64-bit double integers and 32- or 64-bit IEEE floating-point numbers. Hence, the VP is a reasonably good match to most real RISC chips like the Alpha, Mips, ARM, and PowerPC, if somewhat short of registers by today's standards. It supports around 60 simple RISC-like arithmetic, logical, and branching instructions, as well as a few special pseudoinstructions. These latter include TAO, which calls Tao's kernel routines, and LIT, which marks literal data that needs to be translated (e.g., from little-endian to big-endian).

The Tao assembler, VPASM, outputs VPcode that you can run directly or you can convert to native code by manually invoking the appropriate translator. The listing "VPcode at Work" shows a sample of VPASM source code. Currently, Tao Systems has translators for the Intel 286, 386, and 486, the Ixmes T800 and T900 transputers, the Mips R3000, and the ARM 601. The PowerPC and the DEC Alpha are next in the pipeline; it takes around six worker-months to produce a new translator.

The Tao Object Model

Tao is a message-passing operating system whose software model is based on objects, processes, and messages. An object is a bundle of data and code that consumes memory, while a process executes an object and consumes processor time. The Tao hardware model involves multiple processors each with a local memory, connected by a network of communications links. Every processor in this network runs a copy of the Tao kernel and the translator from VPcode to its own native code. Whenever Tao creates a new object, it allocates the object to a processor and then starts a process to execute the object.

All Tao objects are constructed from variable-size blocks of contiguous memory

VPcode at Work A sample tool written in VP assembly language. This tool changes the backdrop picture shown on the Tao's GUI desktop to another bit-map file that you select from a browse window.

```
include "tas.inc"

node bproc,CONTROLTP,0,TEMPLATE
control 0,1624,DATA0P,0,0,0,0
component pblock-bproc
component
bitting pblockNO,"DESK/BACKDROP"
nodeend bproc

node cl_b,CONTROL,VP,TEMPLATE
tool "DESK/BACKDROP"
:fronts
:in:control object pointer
alloctool 80,n?
loop

    :request filename from user
    :op cl_r,0
    :lea both,r1
    :call GET/CL_BROWSE
    :breakif n=0

    :call will be parent
    :inquire "BACKDROP",0
    :breakif n=0

    :end name to backdrop
    :op cl_r,1
    :call LIT/PA_SAVE

endloop
freestack 80
ret

string both,"BITMAPS/*.*BMP"
toolend cl_b
nodeend cl_b

end
```

called nodes. Each node contains two link fields so that the kernel can manage them in doubly linked lists. Nodes can contain data or code, and they have a type field that identifies the type of object they hold. Tao itself doesn't type-check the application of operations to data, although you can implement such type-checking at a higher level within an OOP (object-oriented programming) language.

While stored on disk or in transit over a communications link, nodes exist as unbound templates, but once nodes are loaded into memory, they are converted to "process-ready" form, and it's at this time that any translation of VPcode to native code takes place. The Tao's kernel on a particular processor inserts a process-ready node onto a list of other process-ready objects, from where it can be processed according to the type of object it holds.

Tools, Control Objects, and Classes

Tao's predefined system object types are tools, control objects, bit maps, graphical objects, and class objects, but programmers are free to define new types. A tool is a node containing executable code that can

act on the data contained in an object, to perform calculations or send and receive messages. A control object is the Tao's equivalent of a program, consisting of one or more component tools that are executed in sequence. Control objects are the smallest unit of parallel distribution and execution under Tao. They are not, however, the smallest unit of memory management, because the kernel can retrieve individual tools from disk and make them process-ready. The kernel that creates a new control object distributes its template (using a special load-balancing algorithm) onto some processor, which starts a process to execute the object. When the last component of the control object is finished, the control object closes, and its process terminates. Every component has at least two tools associated with it: one that creates the component and one that cleans up after it dies.

A control object's template contains only the text names of its component tools, not their actual code. When the kernel creates a new control object, it first checks whether any of these specified components are already in memory, and if so, it just points to them—otherwise, it fetches them from disk and makes them process-ready (first translating them, if necessary). Binding under Tao is thus fully dynamic so that no module gets loaded until it is needed and only one copy is ever present in memory. The Tao's processes that execute control objects are lightweight, equivalent to "threads" in operating systems like OS/2, and more than one process can share the same tool's code in multithreaded fashion.

Class objects provide the highest level of organization under Tao. A class encapsulates a group of message-passing objects that can run in parallel, hiding them behind an OOP method interface. Users of classes like Window or PolygonWorld make method calls to the class object (e.g., to open a new window) and are shielded from the complexity of the underlying parallelism that's generated by the execution of the objects hidden within the class.

The Kernel and Memory Management

The simplest version of the Tao kernel is just 12 KB in size and is responsible

for multitasking (via a time-sliced process scheduler), memory management, and the mail and naming systems. Tao Systems is working on a Posix-compliant version of the kernel that implements virtual memory and memory protection on microprocessors that have suitable MMU (memory management unit) hardware, but the 12-KB version does not offer these features.

All executable code in Taos is contained in tools, apart from the small bootstrap loader on each processor that must be in native code. Even the kernel itself is built from tools and is largely written in VP-code. Device drivers are just like any other

process, running outside and in parallel with the kernel. Link drivers running outside the kernel handle all I/O messages, although the kernel handles some local I/O support mechanisms such as data caching.

A process called the Migrator, which runs outside the kernel, is responsible for relocating tool objects in memory and for incremental garbage collection. The lifetime of a Taos tool in memory is determined by its status, according to four different degrees of volatility:

1. Virtual tools are loaded, translated, and bound only when another tool calls them.

The translated code remains in memory until the tool's reference count (which the kernel keeps) falls to zero, after which the kernel can flush the code whenever it needs memory. The kernel may allocate a virtual tool at any time.

2. Nonvirtual tools get loaded and bound at the same time as the tool that references them, and they remain in memory, exempt from relocation, for at least the lifetime of their caller.

3. Semivirtual tools are loaded and bound only when called, like virtual tools, but they then remain in memory like nonvirtual tools.

4. Nonvirtual tools can be flagged as *embedded*, which causes the translator to embed them as in-line code in their caller's code. This is a speed optimization that is extensively used in the kernel code.

Taos Kernel Calls

This selection from among the 64 Taos kernel calls gives some impression of the kind of services that the kernel provides.

Mailbox Management

SENDMAIL	Send a mail message.
COPYMAIL	Copy a mail message then send the copy.
READMAIL	Read a mail message from a mailbox.
READTYPE	Read a mail message from a mailbox; wait until specified type arrives.

Control Object Management

STARTCONTROL	Start a control object locally.
OPENCONTROL	Create a control object and start locally.
OPENCHILD	Create, distribute, and start a control object in the network.
OPENARRAY	Create, distribute, and open a number of different control objects.
OPENFARM	Create, distribute, and open multiple instances of a control object.
OPENDEVICE	Create and transport a control object to a specified processor and start it.
OPENGLOBAL	Create, distribute, and open multiple instances of a control object. Guarantee one control object on every processor.
OPENREMOTE	Create and transport a control object to a specified processor for distribution from that processor, then start it.
OPENPIPE	Create, distribute, and open a pipeline of control objects.

Tool Object Management

VCALL	Virtual call tool object.
OPENTOOL	Request tool object load.
CLOSETOOL	Close tool object.
FLUSHNAMES	Flush named tools from local tool list.
FLUSHTOOLS	Flush nonreferenced tools from local tool list.
UNCLOSETOOL	Increment a tool object's reference count.

General Object Management

VADDR	Obtain address of embedded object.
OBJPROC	Process an object using the existing thread.
LISTPROC	Process a linked list of objects using the existing thread.
LISTTEST	Search list for types of node.

Processor Type ID and Mailbox ID

RINDTYPE	Inquire processor ID of a processor node of specified processor type and with a minimum memory requirement.
GETMYID	Inquire mailbox ID of own control object.
GETPARENT	Inquire mailbox ID of parent control object.
GETSERVER	Inquire server mailbox ID for an object.
NETINFO	Inquire processor and network information.

processor. The typing system enables the kernel to trap system messages (e.g., executable code) and also allows user-defined objects to prioritize the way they read their waiting mail. Objects can employ the READMAIL kernel call to read messages from their mailbox, adding a list of the desired message types as a parameter. The result of such a call might be a message of the required type, or the news that there are no such messages—if the mailbox contains no messages at all then READMAIL suspends the calling process until some mail arrives so that you can use mail messages to awaken sleeping objects.

Taos's link drivers hide the details of the physical transport mechanism that implements the communications links from user programs (although real-time performance issues may sometimes intrude). In the rendering demonstration I mentioned earlier in this article, the transputers were connected via their serial links, while the Mips R3000 chips were connected together through FIFO (first-in/first-out) chips, and all of them talked to the 486 host CPU via the PC's VL-Bus.

Execution in Parallel

Taos is a fully distributed operating system that doesn't attempt to exert control over the execution of parallel applications. Obviously, in practice, you must pick out one processor from which to boot the system, but once all the kernels are booted Taos programs tend to spread out over the network of processors in an almost organic manner, controlled by a distributed load-balancing algorithm. If you examine the "control object management" subsection in



Bearing a passing resemblance to Motif, the Taos GUI is actually a class object that requires just two hardware-specific primitives to work with any display type.

seeking out the lowest points, or gullies, in process-on-loading space.

Each link driver also maintains a table of encoded information about the network topology, used by the kernel to route messages. These tables are dynamically updated at run time so that if a new processor is added to the system, news of its existence spreads outward like a wave. The nature of the routing algorithm reduces the probability of deadlock due to circular message paths, and it can usually find multiple paths between two processors (if they exist), which provides a degree of fault tolerance if a link fails.

A programmer can always override the automatic load balancing and allocate objects to specified processors, by using the OPENDEVICE or OPENGLOBAL calls, while OPENREMOTE invokes a partially automatic mechanism where you explicitly send many objects to a particular processor but let Taos distribute them automatically from there. For example, you could specify that a 1000-process ray-tracing calculation should be run by sending groups of 100 objects to 10 different processors, with Taos completing the distribution.

The Parallel PC

Although Taos can support its own file and display systems, the current release version is PC-hosted, using the MS-DOS file system and an SVGA graphics adapter for display. I received Taos on six 1.44-MB floppy disks, although more than three of these were filled with bit maps and MPEG animations. I was able to run Taos happily on my 486DX266 Elonex PC as a single-processor operating system, coexisting on the same hard disk with Windows (though hardly surprising, it would not run under Windows).

Taos comes with a very simple GUI whose look and feel is loosely modeled on Motif. Control objects that you store in the Taos/Control directory automatically appear on a pop-up menu from where you can execute them with a mouse click. To supplement this GUI, you can open a shell window and use a command-line interface, with a system that resembles DOS. But unlike DOS, Taos command lines represent genuine pipelines in which each successive command launches a separate process whose output is fed to the next.

continued

the text box "Taos Kernel Calls" on page 56, you'll see the kind of services that are available for spawning remote processes. These kernel calls use the mail system to transfer executable VProcs from one processor to another.

Information about the system's performance and current loading is stored in the link drivers that control each communications channel. At boot-up, each link driver benchmarks the processors to which it's attached (by timing the VProc-to-processor). This number is then divided by the number of processes currently running to give a measure of available power for each processor. The automatic load-balancing algorithm uses these power figures in the allocation of new processes. When a tool object arrives at a processor, the local kernel inspects all the links leading outward and asks "is there one of my nearest neighbors who's got more spare power than I have?"—if so, the object is passed on; if not, it executes here.

Applications that dynamically spawn many parallel processes spread out like water running down a mountain, the flow

The Taos Architecture



The Taos operating system runs on a virtual processor. Hardware-specific kernels translate Taos VProcs into native code. The virtual processors communicate using an object-based message-passing system.